Explotando la localidad y tamaños de instancia

Eduardo Alanis García

2026-10-21

Tabla de contenidos

1 Actividad: Explotando la localidad y tamaños de instancia 1
1.1 Introducción . . . 1
1.2 Planteamiento del problema . . . 2
1.3 Desarrollo de las actividades . . . 2
1.3.1 Generación de las instancias . . . 2
1.3.2 Algoritmos de ordenamiento . . . 3
1.3.3 Comprobación de los algoritmos . . . 7
1.3.4 Algoritmos de ordenamiento con contador . . . 8
1.4 Experimentos y resultados . . . 13
1.4.1 Medición de tiempos y comparaciones . . . 13
1.4.2 Gráficas . . . 17
1.4.3 Punto de cruce . . . 21
1.4.4 Discusión . . . 22
1.5 Conclusiones . . . 22
1.6 Referencias . . . 23

1. Actividad: Explotando la localidad y tamaños de instancia

1.1. Introducción

En esta actividad comparo cinco algoritmos de ordenamiento con arreglos aleatorios y casi ordenados de distintos tamaños. Un algoritmo adaptativo ajusta el trabajo que realiza al orden que ya tienen los datos; por eso, puede necesitar menos comparaciones o movimientos cuando la entrada está parcialmente ordenada (Estivill-Castro & Wood, 1992). En este reporte, llamo **localidad** al orden previo de los elementos dentro del arreglo, que represento mediante instancias casi ordenadas. Cook y Kim (1980) observaron que insertion sort es competitivo en listas pequeñas o muy ordenadas, lo que motiva incluirlo en la comparación.

También comparo el número de comparaciones y considero heap sort. El **punto de cruce** es el menor tamaño probado a partir del cual quick sort o merge sort tienen un tiempo mediano menor que insertion sort, tanto en ese tamaño como en todos los tamaños mayores evaluados. Si solo se cumple en el tamaño máximo probado, el resultado es provisional porque no hay tamaños posteriores para confirmarlo.

1.2. Planteamiento del problema

Se ordenan arreglos de tamaños $n \in \{4, 8, 16, 32, 64, 128, 256, 512\}$ en dos condiciones:

1. **Aleatoria:** un generador de datos para crear instancias aleatorias de tamaño variable n.
2. **Casi ordenada:** se parte del arreglo ordenado y se realizan intercambios entre posiciones aleatorias, en una cantidad equivalente al $10\,\%$ del tamaño n.

En ambas condiciones se comparan insertion sort, bubble sort adaptativo, quick sort, merge sort y heap sort en todos los tamaños descritos. Para cada tamaño se registra el tiempo mediano y el número de comparaciones de cada algoritmo. Finalmente, se identifica el punto de cruce de quick sort y merge sort respecto a insertion sort.

1.3. Desarrollo de las actividades

1.3.1. Generación de las instancias

```
using BenchmarkTools
using DataFrames
using Markdown
using Plots
using Random
using Statistics

gr()
default(fmt = :png)

function generar_casi_ordenado(n)
    A = collect(1:n)
    intercambios = round(Int, 0.10 * n)

    for i in 1:intercambios
        a = rand(1:n) # Posición aleatoria
        b = rand(1:n)
        while a == b
```

```
            b = rand(1:n)
        end
        A[a], A[b] = A[b], A[a] # Intercambio
    end

    return A
end

tamaños = 2 .^ (2:9)
Random.seed!(1234)

instancias_aleatorias = [shuffle!(collect(1:n)) for n in tamaños]
instancias_casi_ordenadas = [generar_casi_ordenado(n) for n in tamaños]
```

```
8-element Vector{Vector{Int64}}:
 [1, 2, 3, 4]
 [4, 2, 3, 1, 5, 6, 7, 8]
 [1, 2, 3, 4, 12, 6, 7, 8, 9, 10, 11, 14, 13, 5, 15, 16]
 [1, 2, 31, 26, 10, 6, 7, 8, 9, 5  …  23, 24, 25, 4, 27, 28, 29, 30, 3, 32]
 [41, 2, 3, 56, 5, 6, 7, 8, 9, 10  …  48, 4, 35, 58, 59, 60, 46, 62, 63, 64]
 [1, 75, 3, 4, 5, 6, 7, 53, 9, 10  …  116, 120, 121, 31, 123, 124, 125, 126, 127, 128]
 [1, 2, 3, 89, 5, 210, 228, 8, 9, 10  …  247, 34, 249, 250, 234, 252, 253, 254, 112, 256]
 [1, 2, 3, 4, 5, 6, 437, 8, 9, 39  …  200, 504, 505, 506, 507, 157, 509, 510, 511, 512]
```

Las instancias casi ordenadas usan intercambios entre posiciones distintas. La cantidad se redondea al entero más cercano. Este porcentaje de intercambios es la definición operativa utilizada aquí; no es la misma medida de orden que Cook y Kim (1980) emplearon para comparar sus listas.

1.3.2. Algoritmos de ordenamiento

Las implementaciones de insertion sort, bubble sort adaptativo, quick sort y heap sort son las del capítulo 4 (Téllez, s. f.-a).

1.3.2.1. Insertion sort

```
function insertion_sort!(A)
    n = length(A)
    for i in 2:n
        key = A[i]
```

```
        j = i - 1
        while j >= 1 && A[j] > key
            A[j + 1] = A[j]
            j -= 1
        end

        A[j + 1] = key
    end

    A
end
```

```
insertion_sort! (generic function with 1 method)
```

1.3.2.2. Bubble sort adaptativo

```
function adaptive_bubble_sort!(A)
    n = length(A)

    for i in 1:n-1
        s = 0
        for j in 1:n-i
            if A[j] > A[j + 1]
                s += 1
                A[j], A[j + 1] = A[j + 1], A[j]
            end
        end

        s == 0 && break
    end

    A
end
```

```
adaptive_bubble_sort! (generic function with 1 method)
```

1.3.2.3. Quick sort

```
using Random

function qsort!(A, low=1, high=length(A))
    if low < high
        piv = part!(A, low, high)
        qsort!(A, low, piv - 1)
        qsort!(A, piv + 1, high)
    end

    A
end

function part!(A, low, high)
    ipiv = rand(low:high)
    A[ipiv], A[high] = A[high], A[ipiv]
    piv = A[high]

    i = low - 1

    for j in low:high - 1
        if A[j] < piv
            i += 1
            A[i], A[j] = A[j], A[i]
        end
    end

    ipiv = i + 1
    A[ipiv], A[high] = A[high], A[ipiv]
    ipiv
end
```

```
part! (generic function with 1 method)
```

1.3.2.4. Heapify

```
function heapify!(A, n, i)
    largest = i
    l = 2i
    r = 2i + 1

    if l <= n && A[l] > A[largest]
```

```
        largest = l
    end
    if r <= n && A[r] > A[largest]
        largest = r
    end
    if largest != i
        A[i], A[largest] = A[largest], A[i]
        heapify!(A, n, largest)
    end
end
```

```
heapify! (generic function with 1 method)
```

1.3.2.5. Heap sort

```
function heap_sort!(A)
    n = length(A)

    for i in n ÷ 2 : -1 : 1
        heapify!(A, n, i)
    end
    for i in n : -1 : 2
        A[1], A[i] = A[i], A[1]
        heapify!(A, i - 1, 1)
    end

    A
end
```

```
heap_sort! (generic function with 1 method)
```

1.3.2.6. Merge sort

```
function merge!(A, low, mid, high, temp, menor)
    for i in low:high
        temp[i] = A[i]
    end

    i = low
    j = mid + 1
```

```
    k = low

    while i <= mid && j <= high
        if !menor(temp[j], temp[i])
            A[k] = temp[i]
            i += 1
        else
            A[k] = temp[j]
            j += 1
        end
        k += 1
    end

    while i <= mid
        A[k] = temp[i]
        i += 1
        k += 1
    end
end

function merge_sort!(A, low, high, temp, menor)
    if low < high
        mid = low + (high - low) ÷ 2
        merge_sort!(A, low, mid, temp, menor)
        merge_sort!(A, mid + 1, high, temp, menor)
        merge!(A, low, mid, high, temp, menor)
    end
    return A
end

function merge_sort!(A, menor)
    temp = similar(A)
    return merge_sort!(A, 1, length(A), temp, menor)
end
```

```
merge_sort! (generic function with 2 methods)
```

1.3.3. Comprobación de los algoritmos

Antes de medir, compruebo que cada algoritmo ordene correctamente un arreglo pequeño.

```julia
algoritmos = [
    ("Insertion sort", insertion_sort!),
    ("Bubble sort adaptativo", adaptive_bubble_sort!),
    ("Quick sort", qsort!),
    ("Merge sort", A -> merge_sort!(A, (a, b) -> a < b)),
    ("Heap sort", heap_sort!)
]
```

```
5-element Vector{Tuple{String, Function}}:
 ("Insertion sort", Main.insertion_sort!)
 ("Bubble sort adaptativo", Main.adaptive_bubble_sort!)
 ("Quick sort", Main.qsort!)
 ("Merge sort", var"#20#21"())
 ("Heap sort", Main.heap_sort!)
```

```julia
prueba = [5, 1, 4, 8, 2, 6, 3, 7]
esperado = collect(1:length(prueba))

for (nombre, algoritmo) in algoritmos
    resultado = algoritmo(copy(prueba))
    @assert resultado == esperado
end

println("Todo correcto")
```

```
Todo correcto
```

1.3.4. Algoritmos de ordenamiento con contador

Para contar comparaciones, uso estas funciones que aumentan el contador cada vez que se comparan dos elementos.

1.3.4.1. Funciones para contar comparaciones

```julia
function mayor_contado!(comparaciones, a, b)
    comparaciones[1] += 1
    return a > b
end
```

```
function menor_contado!(comparaciones, a, b)
    comparaciones[1] += 1
    return a < b
end
```

```
menor_contado! (generic function with 1 method)
```

1.3.4.2. Insertion sort

```
function contar_insertion_sort!(A)
    comparaciones = [0]
    n = length(A)

    for i in 2:n
        key = A[i]
        j = i - 1

        while j >= 1 && mayor_contado!(comparaciones, A[j], key)
            A[j + 1] = A[j]
            j -= 1
        end

        A[j + 1] = key
    end

    return comparaciones[1]
end
```

```
contar_insertion_sort! (generic function with 1 method)
```

1.3.4.3. Bubble sort adaptativo

```
function contar_adaptive_bubble_sort!(A)
    comparaciones = [0]
    n = length(A)

    for i in 1:n-1
        s = 0
        for j in 1:n-i
            if mayor_contado!(comparaciones, A[j], A[j + 1])
```

```
                s += 1
                A[j], A[j + 1] = A[j + 1], A[j]
            end
        end

        s == 0 && break
    end

    return comparaciones[1]
end
```

```
contar_adaptive_bubble_sort! (generic function with 1 method)
```

1.3.4.4. Quick sort

```
function part_contar!(A, low, high, comparaciones)
    ipiv = rand(low:high)
    A[ipiv], A[high] = A[high], A[ipiv]
    piv = A[high]
    i = low - 1

    for j in low:high - 1
        if menor_contado!(comparaciones, A[j], piv)
            i += 1
            A[i], A[j] = A[j], A[i]
        end
    end

    ipiv = i + 1
    A[ipiv], A[high] = A[high], A[ipiv]
    return ipiv
end

function qsort_contar!(A, comparaciones, low=1, high=length(A))
    if low < high
        piv = part_contar!(A, low, high, comparaciones)
        qsort_contar!(A, comparaciones, low, piv - 1)
        qsort_contar!(A, comparaciones, piv + 1, high)
    end
end
```

```
function contar_qsort!(A)
    comparaciones = [0]
    qsort_contar!(A, comparaciones)
    return comparaciones[1]
end
```

```
contar_qsort! (generic function with 1 method)
```

1.3.4.5. Merge sort

```
function merge_contar!(A, low, mid, high, temp, comparaciones)
    for i in low:high
        temp[i] = A[i]
    end

    i = low
    j = mid + 1
    k = low

    while i <= mid && j <= high
        if !menor_contado!(comparaciones, temp[j], temp[i])
            A[k] = temp[i]
            i += 1
        else
            A[k] = temp[j]
            j += 1
        end
        k += 1
    end

    while i <= mid
        A[k] = temp[i]
        i += 1
        k += 1
    end

end

function merge_sort_contar!(A, low, high, temp, comparaciones)
    if low < high
        mid = low + (high - low) ÷ 2
```

```
        merge_sort_contar!(A, low, mid, temp, comparaciones)
        merge_sort_contar!(A, mid + 1, high, temp, comparaciones)
        merge_contar!(A, low, mid, high, temp, comparaciones)
    end
end

function contar_merge_sort!(A)
    comparaciones = [0]
    temp = similar(A)
    merge_sort_contar!(A, 1, length(A), temp, comparaciones)
    return comparaciones[1]
end
```

```
contar_merge_sort! (generic function with 1 method)
```

1.3.4.6. Heap sort

```
function heapify_contar!(A, n, i, comparaciones)
    largest = i
    l = 2i
    r = 2i + 1

    if l <= n && mayor_contado!(comparaciones, A[l], A[largest])
        largest = l
    end
    if r <= n && mayor_contado!(comparaciones, A[r], A[largest])
        largest = r
    end

    if largest != i
        A[i], A[largest] = A[largest], A[i]
        heapify_contar!(A, n, largest, comparaciones)
    end
end

function contar_heap_sort!(A)
    comparaciones = [0]
    n = length(A)

    for i in n ÷ 2 : -1 : 1
        heapify_contar!(A, n, i, comparaciones)
```

```
    end
    for i in n : -1 : 2
        A[1], A[i] = A[i], A[1]
        heapify_contar!(A, i - 1, 1, comparaciones)
    end

    return comparaciones[1]
end
```

```
contar_heap_sort! (generic function with 1 method)
```

1.4. Experimentos y resultados

1.4.1. Medición de tiempos y comparaciones

El conteo se realiza por separado del cronometraje. `@benchmark` mide los algoritmos originales, sin el costo adicional de actualizar el contador. El uso de la mediana para resumir las mediciones sigue la metodología de análisis experimental presentada por Téllez (s. f.-b).

1.4.1.1. Ejecución y medición

```
contadores = [
    contar_insertion_sort!,
    contar_adaptive_bubble_sort!,
    contar_qsort!,
    contar_merge_sort!,
    contar_heap_sort!
]

function contar_comparaciones(contador, entrada)
    A = copy(entrada)
    comparaciones = contador(A)
    @assert issorted(A)         # comprueba que A sí quedó ordenado
    return comparaciones
end

function ordenar_lote!(algoritmo, lote)
    for A in lote
        algoritmo(A)
    end
```

```
end

resultados = DataFrame(
    instancia = String[],
    n = Int[],
    algoritmo = String[],
    comparaciones = Int[],
    tiempo_mediano_ns = Float64[],
    muestras = Int[]
)

for (tipo, instancias) in (
    ("Aleatoria", instancias_aleatorias),
    ("Casi ordenada", instancias_casi_ordenadas)
)
    for (n, entrada) in zip(tamaños, instancias)
        for i in 1:length(algoritmos)
            nombre, algoritmo = algoritmos[i]
            comparaciones = contar_comparaciones(contadores[i], entrada)
            tiempo = @benchmark(
                ordenar_lote!($algoritmo, lote),
                setup = (lote = [copy($entrada) for _ in 1:100]),
                samples = 50,
                evals = 1
            )

            push!(resultados, (
                tipo,
                n,
                nombre,
                comparaciones,
                Float64(median(tiempo).time / 100),
                length(tiempo)
            ))
        end
    end
end
```

La tabla 1 resume las comparaciones y los tiempos medianos de los algoritmos con entradas aleatorias.

Tabla 1: Resultados de los algoritmos de ordenamiento en instancias aleatorias.

Tamaño n	Algoritmo	Comparaciones	Tiempo mediano (ns)
4	Insertion sort	5	10.0
4	Bubble sort adaptativo	6	11.0
4	Quick sort	5	53.0
4	Merge sort	5	178.5
4	Heap sort	6	14.0
8	Insertion sort	16	24.0
8	Bubble sort adaptativo	25	35.0
8	Quick sort	15	128.5
8	Merge sort	15	343.5
8	Heap sort	26	60.0
16	Insertion sort	75	63.0
16	Bubble sort adaptativo	119	133.0
16	Quick sort	52	336.5
16	Merge sort	47	722.0
16	Heap sort	81	182.0
32	Insertion sort	257	152.0
32	Bubble sort adaptativo	490	484.5
32	Quick sort	115	816.5
32	Merge sort	119	1488.0
32	Heap sort	219	502.5
64	Insertion sort	951	430.5
64	Bubble sort adaptativo	2015	1891.0
64	Quick sort	338	2259.0
64	Merge sort	300	2873.0
64	Heap sort	574	1605.0
128	Insertion sort	4239	2581.0
128	Bubble sort adaptativo	8113	7442.0
128	Quick sort	897	4559.5
128	Merge sort	740	6000.0
128	Heap sort	1400	3546.5
256	Insertion sort	17200	8777.5
256	Bubble sort adaptativo	32574	29905.0
256	Quick sort	2380	10437.5
256	Merge sort	1722	13293.5
256	Heap sort	3316	8509.5
512	Insertion sort	68229	34482.0
512	Bubble sort adaptativo	130563	165767.0
512	Quick sort	5338	23142.5
512	Merge sort	3980	28150.0

Tamaño n	Algoritmo	Comparaciones	Tiempo mediano (ns)
512	Heap sort	7652	20618.0

La tabla 2 presenta las mismas métricas para las entradas casi ordenadas.

Tabla 2: Resultados de los algoritmos de ordenamiento en instancias casi ordenadas.

Tamaño n	Algoritmo	Comparaciones	Tiempo mediano (ns)
4	Insertion sort	3	8.0
4	Bubble sort adaptativo	3	6.0
4	Quick sort	6	51.0
4	Merge sort	4	148.0
4	Heap sort	7	26.0
8	Insertion sort	10	19.0
8	Bubble sort adaptativo	22	30.0
8	Quick sort	20	137.0
8	Merge sort	13	399.5
8	Heap sort	27	84.5
16	Insertion sort	31	57.0
16	Bubble sort adaptativo	105	196.0
16	Quick sort	59	599.0
16	Merge sort	39	739.0
16	Heap sort	83	195.0
32	Insertion sort	138	105.0
32	Bubble sort adaptativo	493	465.0
32	Quick sort	142	690.5
32	Merge sort	108	1415.5
32	Heap sort	225	544.0
64	Insertion sort	343	238.5
64	Bubble sort adaptativo	1961	1477.0
64	Quick sort	335	1553.0
64	Merge sort	276	2920.5
64	Heap sort	587	1541.5
128	Insertion sort	1008	834.5
128	Bubble sort adaptativo	7498	5577.0
128	Quick sort	945	3438.5
128	Merge sort	619	5662.0
128	Heap sort	1457	3857.0
256	Insertion sort	3649	2547.5
256	Bubble sort adaptativo	32079	28466.5
256	Quick sort	2468	7740.5

Tamaño n	Algoritmo	Comparaciones	Tiempo mediano (ns)
256	Merge sort	1543	13117.0
256	Heap sort	3422	8920.5
512	Insertion sort	13626	9033.0
512	Bubble sort adaptativo	127576	112175.5
512	Quick sort	5761	16918.0
512	Merge sort	3656	27003.0
512	Heap sort	7878	21788.5

1.4.2. Gráficas

Las Figuras figura 1, figura 2, figura 3 y figura 4 muestran el tiempo mediano y el número de comparaciones para ambas condiciones, en los ocho tamaños probados.

```
function graficar_resultado(tabla, tipo, metrica, etiqueta, titulo)
    p = plot(
        xscale = :log2,
        yscale = :log10,
        xlabel = "Tamaño n",
        ylabel = etiqueta,
        title = "$titulo: $tipo",
        legend = :topleft
    )

    for (nombre, algoritmo) in algoritmos
        datos = tabla[(tabla.instancia .== tipo) .& (tabla.algoritmo .== nombre), :]
        plot!(
            p,
            datos.n,
            datos[!, metrica],
            label = nombre,
            marker = :circle,
            linewidth = 2
        )
    end

    return p
end
```

```
graficar_resultado (generic function with 1 method)
```

1.4.2.1. Tiempos de las instancias aleatorias

La figura 1 compara los tiempos medianos de los cinco algoritmos en las instancias aleatorias.

```
grafica_tiempos_aleatoria = graficar_resultado(
    resultados, "Aleatoria", :tiempo_mediano_ns, "Tiempo mediano (ns)", "Tiempo de ordenamien
)
display(grafica_tiempos_aleatoria)
```

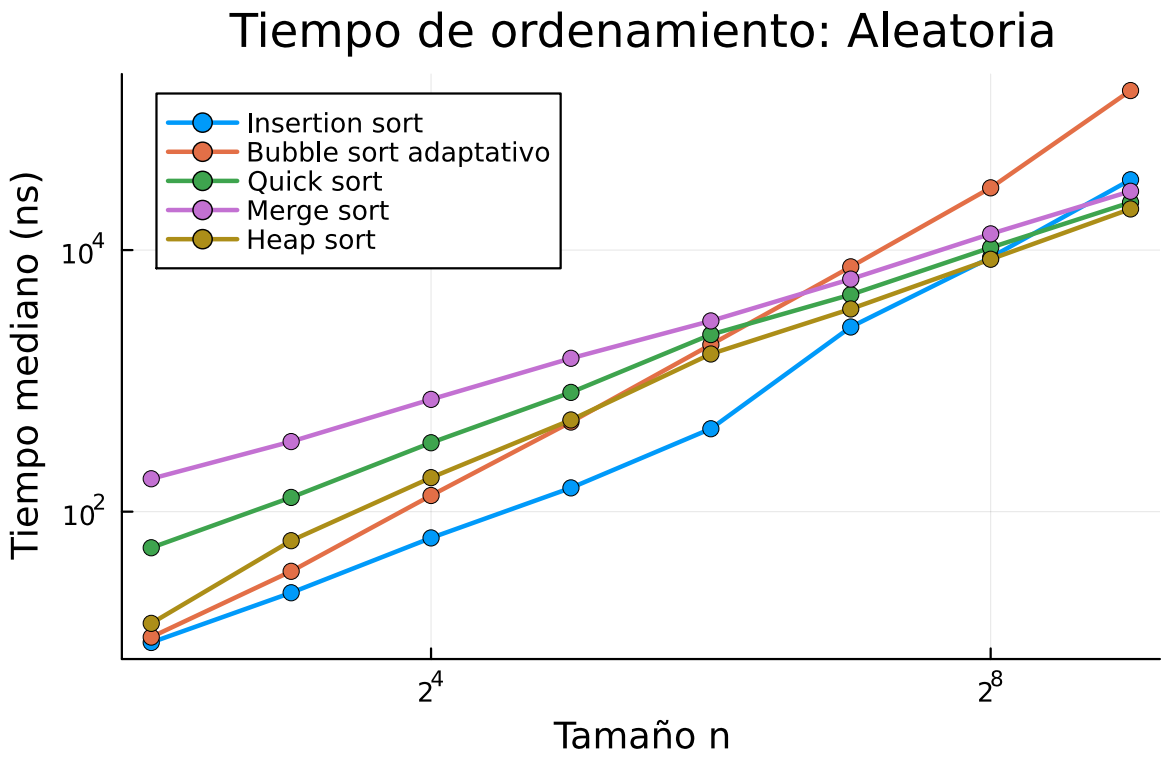


Figura 1: Tiempo mediano de ordenamiento para instancias aleatorias.

1.4.2.2. Comparaciones de las instancias aleatorias

La figura 2 compara el número de comparaciones en las instancias aleatorias.

```
grafica_comparaciones_aleatoria = graficar_resultado(
    resultados, "Aleatoria", :comparaciones, "Número de comparaciones", "Comparaciones"
)
display(grafica_comparaciones_aleatoria)
```

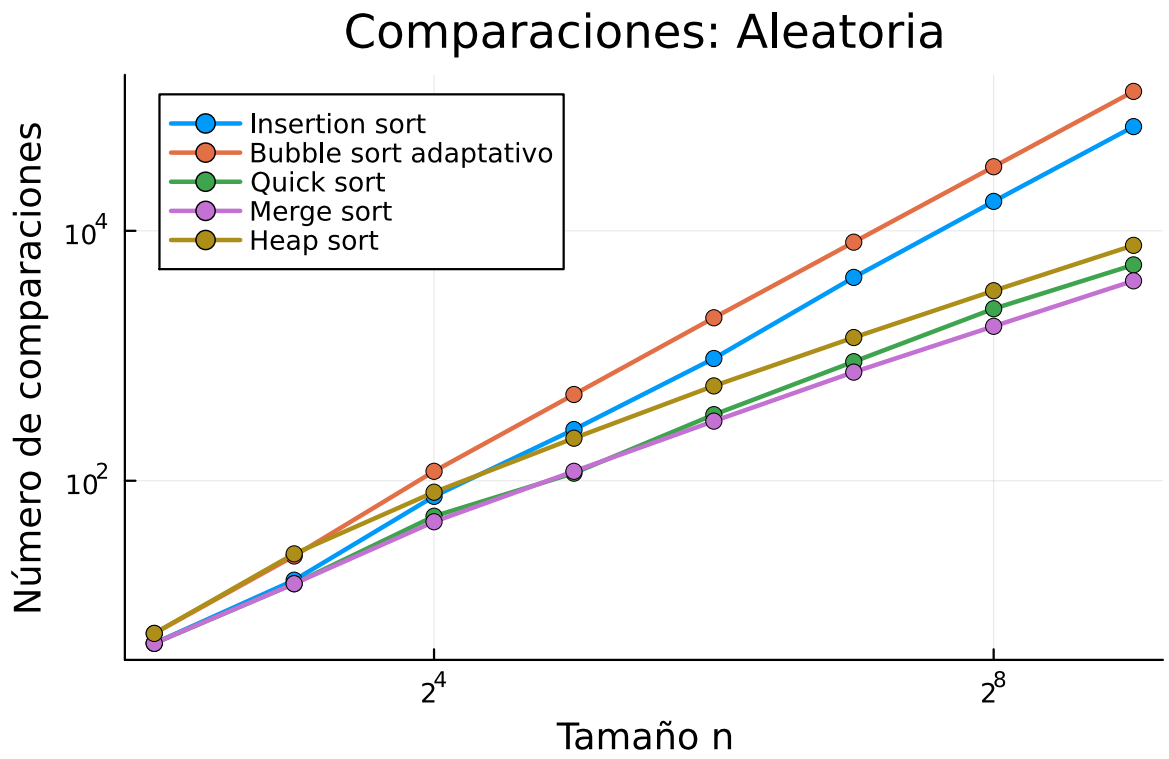


Figura 2: Número de comparaciones para instancias aleatorias.

1.4.2.3. Tiempos de las instancias casi ordenadas

La figura 3 compara los tiempos medianos de los cinco algoritmos en las instancias casi ordenadas.

```
grafica_tiempos_casi_ordenada = graficar_resultado(
    resultados, "Casi ordenada", :tiempo_mediano_ns, "Tiempo mediano (ns)", "Tiempo de ordena
)
display(grafica_tiempos_casi_ordenada)
```

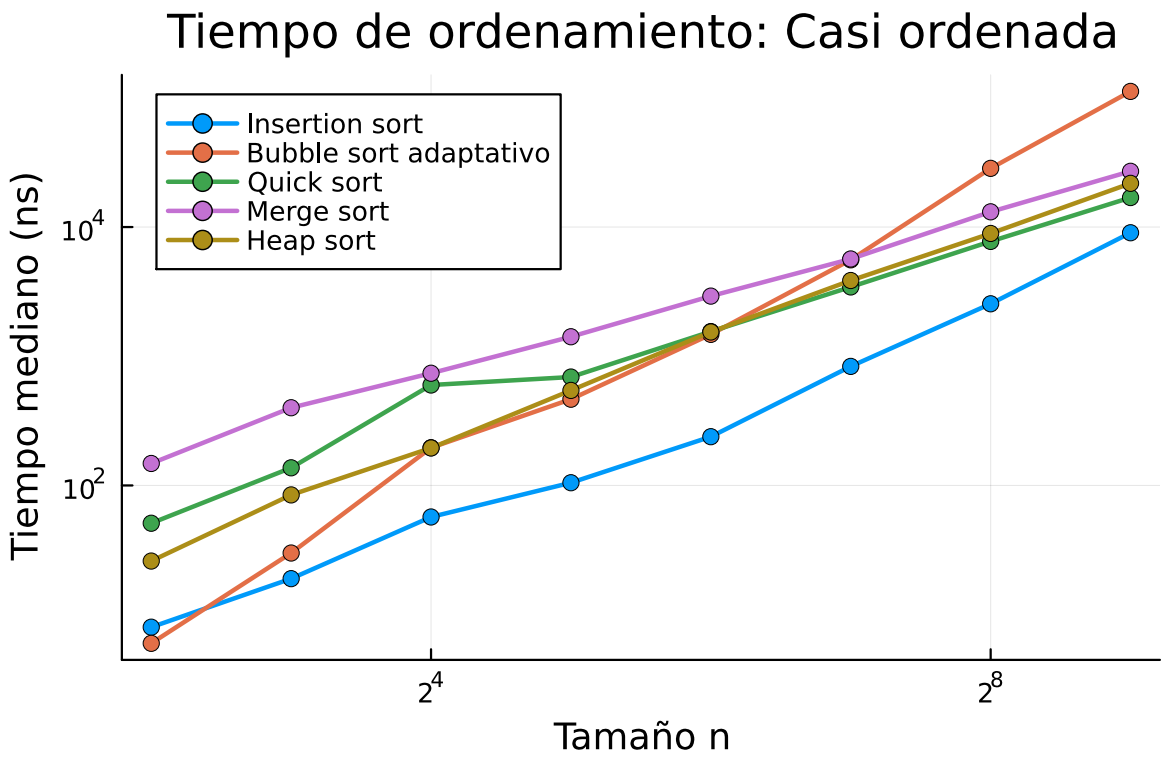


Figura 3: Tiempo mediano de ordenamiento para instancias casi ordenadas.

1.4.2.4. Comparaciones de las instancias casi ordenadas

La figura 4 compara el número de comparaciones en las instancias casi ordenadas.

```
grafica_comparaciones_casi_ordenada = graficar_resultado(
    resultados, "Casi ordenada", :comparaciones, "Número de comparaciones", "Comparaciones"
)
display(grafica_comparaciones_casi_ordenada)
```

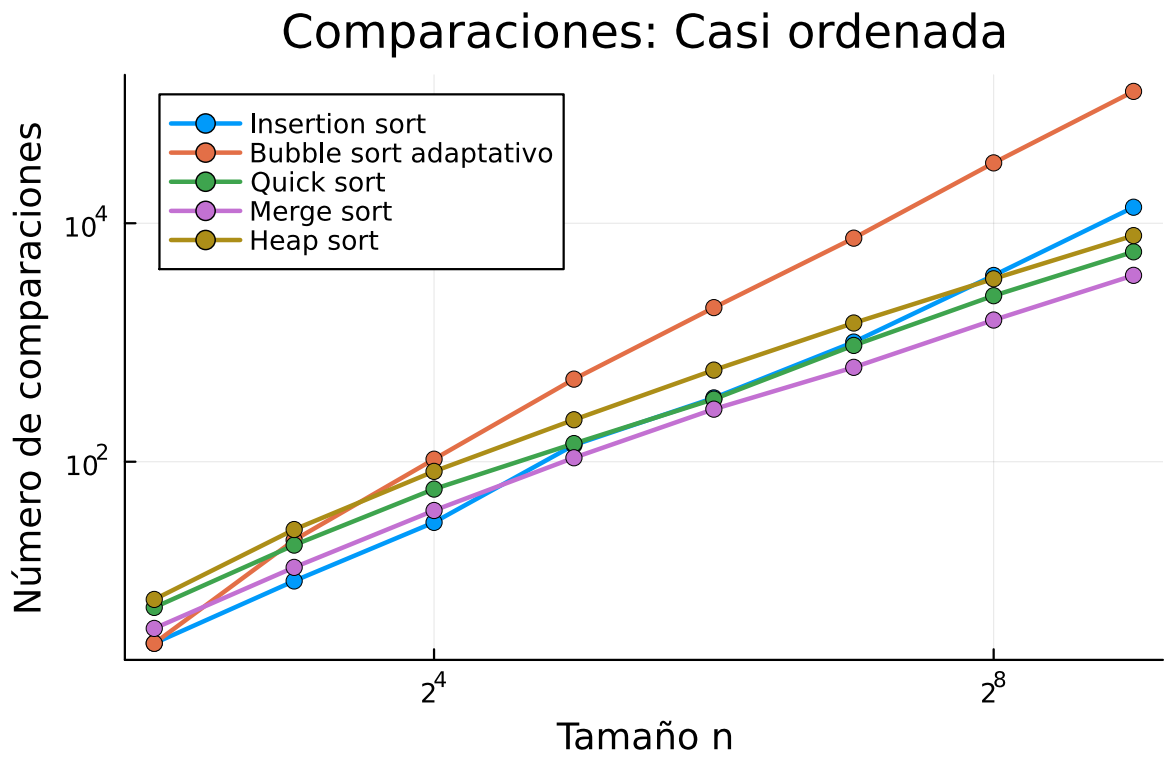


Figura 4: Número de comparaciones para instancias casi ordenadas.

1.4.3. Punto de cruce

Para cada condición, busco el primer tamaño en el que quick sort o merge sort tiene un tiempo mediano menor que insertion sort y conserva esa ventaja en todos los tamaños evaluados siguientes. La tabla 3 resume los puntos encontrados; un resultado de $n = 512$ es provisional porque ese fue el tamaño máximo probado.

Tabla 3: Puntos de cruce de quick sort y merge sort respecto a insertion sort.

Algoritmo	Aleatoria	Casi ordenada	Cambio en n
Quick sort	n = 512*	No confirmado	Sin cruce casi ordenada
Merge sort	n = 512*	No confirmado	Sin cruce casi ordenada

En la tabla 3, el cambio indica cuánto varió el tamaño del punto de cruce entre las dos condiciones. “Sin cruce” significa que no se encontró uno entre los tamaños probados. El asterisco señala que $n = 512$ es el último tamaño evaluado.

1.4.4. Discusión

Complejidad y resultados en instancias aleatorias. Insertion sort y bubble sort adaptativo tienen tiempo $O(n^2)$ en el peor caso; cuando la entrada ya está ordenada, ambos pueden acercarse a $O(n)$, aunque por mecanismos distintos. Merge sort y heap sort tienen tiempo $O(n \log n)$, mientras que quick sort tiene tiempo esperado $O(n \log n)$ con pivotes aleatorios y un peor caso de $O(n^2)$. Estos órdenes describen cómo crece el trabajo al aumentar n.

En las entradas aleatorias de tamaño 512, insertion sort hizo 68,229 comparaciones y tardó 40,754.5 ns. Quick sort hizo 5,338 comparaciones y tardó 51,207.5 ns; merge sort hizo 3,980 y tardó 63,211 ns. Aunque estos dos últimos realizan asintóticamente menos trabajo que insertion sort, sus tiempos fueron mayores en este rango. No contradice la complejidad asintótica. En esta corrida, la tabla 3 no confirma un cruce hasta 512.

Constantes y localidad de memoria. Insertion sort recorre posiciones contiguas y desplaza elementos cercanos dentro del mismo arreglo; usa poca memoria adicional y tiene un flujo de control simple. Esa localidad y el bajo costo por operación ayudan a explicar por qué puede ser competitivo en tamaños pequeños y moderados, incluso cuando hace más comparaciones. Quick sort también particiona segmentos contiguos en el arreglo, aunque añade recursión y selección aleatoria del pivote. Merge sort recorre secuencialmente sus mitades, lo que favorece accesos contiguos, pero copia elementos a un arreglo auxiliar. Heap sort usa espacio auxiliar constante, pero salta entre un nodo y sus hijos en el arreglo, un patrón menos local que un recorrido secuencial.

La tabla 1 también muestra que heap sort no fue siempre más rápido que quick sort: por ejemplo, en $n = 32$ quick sort tardó 1,056.5 ns y heap sort 1,137.5 ns. En $n = 512$, heap sort sí fue más rápido que quick sort y merge sort, pero no que insertion sort. Por tanto, los resultados no respaldan un orden fijo de rapidez entre todos los algoritmos; muestran cómo interactúan la complejidad, las constantes, la localidad y el tamaño de la instancia.

1.5. Conclusiones

- Insertion sort fue más rápido que quick sort y merge sort hasta el tamaño máximo probado ($n = 512$), tanto con entradas aleatorias como casi ordenadas. No se confirmó un punto de cruce dentro del intervalo evaluado.
- En las instancias casi ordenadas no se encontró un punto de cruce en el intervalo $n \in \{4, \dots, 512\}$. Si existe uno, ocurre después de 512; estos datos no permiten estimar en qué tamaño.
- La ventaja asintótica de los algoritmos $O(n \log n)$ no garantizó menor tiempo hasta $n = 512$. En ese tamaño, insertion sort hizo muchas más comparaciones en la instancia aleatoria, pero aun así tardó menos que quick sort y merge sort.

Nota: Los tiempos y, algunos conteos pueden variar al volver a ejecutar el experimento. Las conclusiones describen la corrida realizada previamente al generar el pdf; nuevos resultados podrían modificar el punto de cruce observado (Razón por la cual me costo trabajo un poco la redaccion de las conclusiones dado que con lo renderizado habia modificaciones en los resultados dado que en algunas veces existian cruces y en otras no).

1.6. Referencias

Cook, C. R., & Kim, D. J. (1980). Best sorting algorithm for nearly sorted lists. *Communications of the ACM, 23*(11), 620–624. https://doi.org/10.1145/359024.359026

Estivill-Castro, V., & Wood, D. (1992). A survey of adaptive sorting algorithms. *ACM Computing Surveys, 24*(4), 441–476. https://doi.org/10.1145/146370.146381

Téllez, E. S. (s. f.-a). *Algoritmos de ordenamiento* (capítulo 4). Análisis de algoritmos con Julia. https://sadit.github.io/ALGO-IR/cap4-ordenamiento.html

Téllez, E. S. (s. f.-b). *Introducción al análisis de algoritmos con Julia* (capítulo 2). Análisis de algoritmos con Julia. https://sadit.github.io/ALGO-IR/cap2-analisis.html